

A Geometric Consistency Theorem for a Symbolic Perturbation Scheme

CHEE-KENG YAP*

*Courant Institute of Mathematical Sciences,
New York University, New York, New York*

Received November 28, 1988; revised March 27, 1989

In a previous paper, we introduced a generic solution to the problem of data degeneracy in geometric algorithms. The scheme is simple to use: algorithms qualifying under our requirements just have to use a prescribed blackbox for polynomial evaluation in order to achieve a symbolic perturbation of data. In this paper, we introduce the concept of an infinitesimal perturbation and show that our method is consistent relative to such perturbations. © 1990 Academic Press, Inc.

1. INTRODUCTION

Theoretical algorithms are a minor industry in computer science that need not necessarily bear any relation to the algorithms in use. This is not necessarily a failing of either area: in some sense, implemented algorithms are expected to lag behind the vanguard "paper" algorithms. The hope is that paper algorithms would eventually be implemented. Even failing this, such algorithms yield valuable insights into the inherent complexity of the problems and the algorithmic techniques may find applications. One reason many paper algorithms are not implemented is the complexity of their implementation, i.e., the constants involved in the big-oh analysis are truly big. In computational geometry (as opposed to combinatorial algorithms, say) we face two additional barriers in the transition from paper to practical algorithms: the issues of *fixed precision computation* (most paper algorithms are described for exact, arbitrary precision computations) and *geometric degeneracy* (paper algorithms generally neglect degenerate cases). The neglect of these issues is often a source of complaints especially by implementors of these algorithms. We defend the general approach that theoreticians have taken in their usual approach to algorithms because attempting to deal directly with either or both issues in any particular algorithm need not necessarily help the implementator: it could have the opposite effect of making the algorithm unimplementable or incom-

* This work was supported by the Deutsche Forschungs Gemeinschaft, SFB 124, TP B2, at the Universitaet des Saarlandes; it is also supported in part by NSF Grants DCR-84-01898 and CCR-8703458.

prehensible. (We do not rule out special situations where one must do precisely both of these unpleasant tasks.) Nevertheless it falls upon the theoreticians (not practitioners) to come up with solutions that can be pipe-fitted with the usual theoretical algorithms, hopefully in a transparent way. In short, we suggest that the proper way to tackle the precision and degeneracy issues is to find *generic solutions*. In [13] we proposed one generic solution for the degeneracy problem, based on symbolic perturbation of data. In this paper we will prove a consistency theorem for our method.

Among the first to discuss explicitly symbolic ways to deal with degeneracy in the context of computational geometry is Edelsbrunner and his co-workers [6, 8, 7]. Actually such an approach is foreshadowed by the “symmetric breaking” methods in the simplex algorithm, going as far back as 1952 (see [3, 4]).

In Section 2, we review the basic results in [13]. In Section 3, we introduce the concept of infinitesimal perturbation and show that our scheme is consistent vis-à-vis such perturbations. In Section 4, we discuss the following implication of the consistency theorem: *any algorithm that is correct with respect to non-degenerate inputs will be correct with respect to all inputs*. Section 5 extends our method to rational functions. We conclude in Section 6.

2. REVIEW OF A PERTURBATION SCHEME

In this section, we review the basic facts from [13]. We make the following fundamental assumption on algorithms to which the method is applied:

- The input to the algorithm consists of a sequence of real numbers $a = (a_1, a_2, \dots, a_n)$ (the *input parameters*). Here, n is unbounded and measures the size of the input.
- The algorithm makes decision steps by way of evaluating polynomials taken from a fixed set D (the *test polynomials*), where *the input parameters are substituted for variables in the test polynomials*. The algorithm makes a 3-way branch depending on the sign of the evaluation. The set D is determined by the algorithm and is independent of the input.
- The input is deemed *degenerate* if a substitution into a test polynomial evaluates to zero in the course of the algorithm.

The set D is often even a finite set. It is crucial to understand the requirement that the substitution for the variables in test polynomials must be by input parameters. This means that values computed from the input parameters cannot be used for the substitution. Many algorithms in computational geometry satisfy these assumptions. For instance, consider algorithms for Voronoi diagrams and algorithms for computing arrangement of hyperplanes. Examples where these assumptions might be violated are (a) if the input includes additional non-numerical structures (such as graphs), (b) where the branching tests involved

substituting computed values for the variables. A case of (a) would be the planar point location problem where one is given a graph structure as well as numerical parameters, and (b) is illustrated by shortest path problems where computed values (distances, obtained as sums of square roots of input values) are used in the branching tests. But even where these assumptions fail, we can sometimes apply our method by simple modifications. For instance, if the computed values that are substituted into the test polynomials are all polynomial functions of the inputs, then we simply have to enlarge the test set D .

Our generic solution is extremely simple and seems eminently implementable. It amounts to providing a blackbox for evaluation of polynomials. The unusual feature of this evaluation is that, we return not just the value of the evaluation, but a non-zero sign (+ or -) as well. This sign will be the expected thing (i.e., the sign of the value) when the value is non-zero. Using such a blackbox, the algorithm can always avoid the zero-branch in decision steps.

Let us see what this blackbox amounts to. It takes as input a polynomial $p(\mathbf{x}) = p(x_1, \dots, x_n)$ (p not identically zero) and a point $\mathbf{a} = (a_1, \dots, a_n)$, and outputs a value-sign pair $(p(\mathbf{a}), s)$ where s is + or -, and such that if $p(\mathbf{a}) \neq 0$ then $\text{sign}(p(\mathbf{a})) = s$.

We need some terminology: let $\mathbf{R} = \mathcal{Q}[x_1, \dots, x_n]$ be a polynomial ring where $\mathcal{Q}$ are the rational numbers. Let $\text{PP} = \text{PP}(x_1, \dots, x_n)$ denote the set of all *power products*,

$$w = \prod_{i=1}^n x_i^{e_i} \quad (e_i \geq 0).$$

The *total degree* of w is $\deg(w) = \sum_{i=1}^n e_i$. In particular, if all the exponents e_i are zero then $w = 1$. For any polynomial $p \in \mathbf{R}$, let

$$p_w \quad \text{or} \quad D_w(p)$$

denote the w th derivative of p . E.g., if $w = x_1^2 x_2$ then

$$p_w = D_w(p) = \frac{\partial^3 p}{\partial^2 x_1 \partial x_2}.$$

An *admissible ordering* $\leq_A$ on PP is a total ordering on PP satisfying (1) $1 \leq_A w$ and (2) $w \leq_A w'$ implies $vw \leq_A vw'$ for all $w, w', v \in \text{PP}$. Two important examples of admissible orderings are:

(A) the (pure) lexicographical orderings $\leq_{\text{LEX}}$ where we compare two power products by the vectors of their exponents, using the usual lexicographical method.

(B) the total degree (lexicographical) ordering $\leq_{\text{TOT}}$ where we compare two power products by first comparing their total degrees and if that is indecisive, then comparing them lexicographically as in (A).

Remark. Interestingly, admissible orderings arise in a totally different context (the theory of Gröbner bases), although we do not suspect any deeper connection. See [1, 9] for an introduction to Gröbner bases. Robbiano [12, 11] had provided a characterization of all admissible orderings; see [5] for another proof. This characterization will be central in our main construction.

Henceforth, we fix any admissible ordering $\leq_A$ on PP. Now for any polynomial p , let $S(p)$ denote the infinite sequence

$$S(p) := (p_{w_0}, p_{w_1}, p_{w_2}, \dots),$$

where $w_0 <_A w_1 <_A w_2 <_A \dots$ lists (exhaustively) all power products in PP. Note that $S(p)$ has only a finite number of non-zero entries. Also, $w_0 = 1$ so $p_{w_0} = p$. The blackbox (on input p and $\mathbf{a}$) evaluates $p_{w_i}(\mathbf{a})$ for $i = 0, 1, \dots$ in succession until the first non-zero value. Since p is not identically zero, we will find a non-zero value. The algorithm then returns the pair $(p(\mathbf{a}), s)$, where s is the sign of the first non-zero evaluated. Hence, for any choice of $\leq_A$ and any choice of $\mathbf{a}$, our method gives rise to a *sign function*,

$$\sigma: \mathbf{R} \rightarrow \{-1, 0, +1\},$$

where $\sigma(p)$ is the sign given by our evaluation scheme on input p and $\mathbf{a}$. We say σ is *induced* by $(\leq_A, \mathbf{a})$.

A *sign function* σ , in general, is a function mapping each element $p \in \mathbf{R}$ to $\sigma(p) \in \{-1, 0, +1\}$ satisfying the axioms

- (i) $\sigma(0) = 0$,
- (ii) $\sigma(p) = -\sigma(-p)$,
- (iii) $\sigma(pq) = \sigma(p)\sigma(q)$,
- (iv) $\sigma(p) = \sigma(q) = +1$ implies $\sigma(p+q) = +1$.

We say the sign function is *total* if (i) is replaced by:

- (i)' $\sigma(p) = 0$ if and only if $p = 0$.

Various properties of σ are described in [13], but it suffices to say here that we can define a total ordering $\leq_\sigma$ on $\mathbf{R}$ using a total sign function:

$$p \underset{\sigma}{>} q \Leftrightarrow \sigma(p - q) = +1.$$

A polynomial ring $\mathbf{R}$ together with a total sign function is called an *ordered ring*. It is shown in [13] that our evaluation scheme (for any choice of admissible ordering and any $\mathbf{a}$) makes $\mathbf{R}$ into an ordered ring.

Remark. The total ordering induced by σ on $\mathbf{R}$ gives rise (by restriction) to a total ordering on $\text{PP} \subseteq \mathbf{R}$; but this ordering on PP is not necessarily the same as the admissible ordering on PP that we started out with.

In implementation, we must realize that $S(p)$ is computed on the fly. This means that we should be able to generate successive entries of $S(p)$ easily. This is determined by the choice of the admissible ordering. In case of common orderings such as lexicographical or total degree orderings, or if the admissible ordering is represented by its weight matrix [12, 5], then this is easily done.

3. INFINITESIMAL PERTURBATION AND CONSISTENCY

The basic justification of our symbolic perturbation scheme in [13] had been the demonstration that the scheme gives rise to total sign functions. This “algebraic reasoning” is somewhat unsatisfactory: we now motivate the need for a more geometric alternative.

Suppose we are given m points $\mathbf{a}_1, \dots, \mathbf{a}_m$ ($\mathbf{a}_i \in \mathcal{R}^n$ for $i = 1, \dots, m$, $\mathcal{R}$ are the reals), all lying in a line. Let Δ_{ijk} , $1 \leq i < j < k \leq m$, be the determinant on $\mathbf{a}_i, \mathbf{a}_j, \mathbf{a}_k$ whose value is zero if and only if $\mathbf{a}_i, \mathbf{a}_j, \mathbf{a}_k$ are colinear; if the value of the determinant is non-zero, then the sign of Δ_{ijk} determines the relative disposition of the three points. Given the set U of all $h = \binom{m}{3}$ determinants, consider the set of 2^h (non-degenerate) sign assignments $\sigma: U \rightarrow \{+, -\}$. Call an assignment σ *realizable* if there is a perturbation of the m points so that σ is achieved. It is not clear that our perturbation scheme may not give rise to unrealizable sign assignments (problem of consistency) or that every realizable sign assignment can be achieved by a suitable choice of admissible ordering (problem of completeness). We will prove a consistency result in this section. In the final section, we give an example showing that our scheme is not complete.

Given a point $\mathbf{a} \in \mathcal{R}^n$ and a set $U \subseteq \mathbf{R}$ of polynomials, the idea of a “ U -perturbation at $\mathbf{a}$ ” is to choose a sufficiently small $\mathbf{d} \in \mathcal{R}^n$ such that

- (i) $p(\mathbf{a})$ and $p(\mathbf{a} + \mathbf{d})$ have the same sign whenever $p(\mathbf{a}) \neq 0$, and
- (ii) $p(\mathbf{a} + \mathbf{d}) \neq 0$ for all $p \in U$.

If U is the entire ring $\mathbf{R}$ then it is not hard to see that there cannot exist a choice of $\mathbf{d}$ to achieve this perturbation. Yet our evaluation scheme appears to be a perturbation of the entire set $\mathbf{R}$. The resolution of this paradox must therefore lie in choosing a non-standard (infinitesimal) $\mathbf{d}$. This is precisely our solution although we prefer a more direct and informative approach instead of invoking non-standard analysis.

DEFINITION. An *infinitesimal perturbation* at $\mathbf{a} \in \mathcal{R}^n$ is a sequence

$$\delta = (\delta(1), \delta(2), \delta(3), \dots), \quad \delta(t) \in \mathcal{R}^n$$

such that

1. $\delta(t) \rightarrow 0_n = (0, 0, \dots, 0)$ as $t \rightarrow \infty$.

2. For every non-zero polynomial p in $\mathbf{R}$, there is a sign, $\text{sign}_\delta(p) \in \{+, -\}$, such that for all but finitely many values of t ,

$$\text{sign}(p(\mathbf{a} + \delta(t))) = \text{sign}_\delta(p).$$

It turns out that our proof of consistency requires the polynomial ring $\mathbf{R}$ to be denumerable; this is the case if the coefficients are rational numbers. In that case, condition 2 in the above definition is equivalent to the following, which we use in the main proof below:

2'. There exists an enumeration¹ Σ of the non-zero polynomials $\mathbf{R} - \{0\}$,

$$\Sigma = (p_1, p_2, p_3, \dots)$$

where for all $t \geq 1$, for all $i \geq t$,

$$\text{sign}(p_i(\mathbf{a} + \delta(t))) = \text{sign}(p_i(\mathbf{a} + \delta(i))) \neq 0.$$

We say that Σ is an *acceptable enumeration* for δ . In our proof, it turns out that δ can be assumed to be *positive*; that is, each component of $\delta(t)$ is positive for all t . We define a function sign_δ such that if $\Sigma = (p_1, p_2, \dots)$ then $\text{sign}_\delta(p_i) = \text{sign}(p_i(\mathbf{a} + \delta(i)))$. The following is easy.

LEMMA 1. *The function sign_δ is well defined (i.e., it does not depend on the choice of the acceptable enumeration Σ). The function sign_δ is a total sign function consistent with evaluation at $\mathbf{a}$, i.e., for all $p \in \mathbf{R}$, $p(\mathbf{a}) \neq 0$ implies $\text{sign}(p(\mathbf{a})) = \text{sign}_\delta(p)$.*

Sign functions arising from infinitesimal perturbations will be called *infinitesimal sign functions*. Let $\leq_\delta$ denote the total order on $\mathbf{R}$ induced by sign_δ . As far as infinitesimal sign functions are concerned, any enumeration can be acceptable:

LEMMA 2. *Let Σ be any enumeration of $\mathbf{R} - \{0\}$ and let σ be any infinitesimal sign function. There is an infinitesimal perturbation δ such that Σ is acceptable for δ and $\text{sign}_\delta = \sigma$.*

The proof is omitted here since this result is not needed later.

We now prove the main result. A key ingredient of our proof is the characterization of admissible orderings [12, 5].

THEOREM 3. (Consistency). *Let $\mathbf{R} = K[x_1, \dots, x_n]$ be a denumerable polynomial ring where K is a subfield of $\mathcal{R}$. Let $\leq_A$ be any admissible ordering and $\mathbf{a} \in \mathcal{R}^n$. If σ is the sign function on $\mathbf{R}$ induced by $(\leq_A, \mathbf{a})$ using our evaluation scheme, then there exists an infinitesimal perturbation δ at $\mathbf{a}$ such that $\sigma = \text{sign}_\delta$.*

¹ An enumeration of a set is a list indexed by the positive integers such that each list item belongs to the set and each element of the set appears at least once on the list.

Proof. Let $\leq_\sigma$ denote the total ordering on $\mathbf{R}$ determined by σ . For every admissible ordering $\leq_A$ we can find an n -square matrix $W = (w_{ij})$ with real entries such that if $u = \prod_{i=1}^n x_i^{d_i}$ and $v = \prod_{i=1}^n x_i^{e_i}$ then

$$u \leq_A v \quad \text{iff} \quad W \cdot \mathbf{d} \leq_{LEX} W \cdot \mathbf{e},$$

where $\mathbf{d} = (d_1, \dots, d_n)^t$, $\mathbf{e} = (e_1, \dots, e_n)^t$ (here $(\cdot)^t$ denotes matrix transpose). We can assume $w_{ij} \geq 0$ (see [5]). We also write the components of the vector $W \cdot \mathbf{d}$ as

$$W \cdot \mathbf{d} = (W_1(u), \dots, W_n(u)). \quad (1)$$

Now let $\Sigma = (p_1, p_2, \dots)$ be any enumeration of polynomials in $\mathbf{R} - \{0\}$. The theorem follows from constructing the desired δ with Σ as an acceptable enumeration. In order that $\sigma = \text{sign}_\delta$, the following property must hold for each $t \geq 1$:

$$\sigma(p_t) = \text{sign}(p_t(\mathbf{a} + \delta(t))). \quad (2)$$

As usual, let

$$S(p_t) = (D_{u_1}(p_t), D_{u_2}(p_t), \dots)$$

denote partial derivatives of p_t listed according to the admissible ordering on power products, $1 = u_1 <_A u_2 <_A \dots$. Let $k = k(t)$ be the first index such that $D_{u_k}(p_t)(\mathbf{a}) \neq 0$. Since $\sigma(p_t)$ is given by the sign of $D_{u_k}(p_t)(\mathbf{a})$, (2) amounts to the goal of showing

$$\text{sign}(D_{u_k}(p_t)(\mathbf{a})) = \text{sign}(p_t(\mathbf{a} + \delta(t))). \quad (3)$$

In the rest of this proof, we fix t and let $p = p_t$. Consider the Taylor expansion of p at $\mathbf{a} + \delta(t)$,

$$p(\mathbf{a} + \delta(t)) = \sum_{j=0}^{\infty} \frac{\Delta^j(p)(\mathbf{a})}{j!}, \quad (4)$$

where

$$\begin{aligned} \Delta &= \left(\delta_1(t) \frac{\partial}{\partial x_1} + \delta_2(t) \frac{\partial}{\partial x_2} + \dots + \delta_n(t) \frac{\partial}{\partial x_n} \right) \\ &= (\delta_1(t) D_{x_1} + \delta_2(t) D_{x_2} + \dots + \delta_n(t) D_{x_n}) \end{aligned}$$

and $\delta(t) = (\delta_1(t), \dots, \delta_n(t))$. Expand Δ^j as

$$\Delta^j = \sum_u u(\delta(t)) C_u D_u, \quad (5)$$

where u range over all power products with total degree $\deg(u) = j$, and C_u is the multinomial coefficient determined by u . More explicitly, if

$$u = u(x_1, \dots, x_n) = \prod_{i=1}^n x_i^{e_i} \quad (6)$$

then

$$u(\delta(t)) = u(\delta_1(t), \dots, \delta_n(t)) := \prod_{i=1}^n \delta_i(t)^{e_i}$$

and

$$C_u := \frac{j!}{e_1! e_2! \dots e_n!}.$$

Rewriting the Taylor expansion (4) using (5),

$$p(\mathbf{a} + \delta(t)) = \sum_{j \geq 1} u_j(\delta(t)) \frac{C_{u_j}}{\deg(u_j)!} D_{u_j}(p)(\mathbf{a}), \quad (7)$$

where $u_1 <_A u_2 <_A \dots$ is the list of power products ordered by the admissible ordering. Call i the *index* of the power product u_i in this list. By our choice of $k = k(t)$ as the smallest index satisfying (3), we get $D_{u_j}(p)(\mathbf{a}) = 0$ for $j < k$. Suppressing double subscripts, the expansion (7) becomes

$$p(\mathbf{a} + \delta(t)) = \sum_{j \geq k} u_j(\delta(t)) c_j D_j(p)(\mathbf{a}), \quad (8)$$

where $c_j := C_{u_j}/\deg(u_j)! > 0$ and D_j is D_{u_j} . It follows that our goal (3) is a consequence of

$$u_k(\delta(t)) c_k \cdot |D_k(p)(\mathbf{a})| > \sum_{j > k} u_j(\delta(t)) c_j \cdot |D_j(p)(\mathbf{a})|. \quad (9)$$

Strictly speaking, we should have used the absolute values $|u_j(\delta(t))|$ (instead of $u_j(\delta(t))$) in (9). However, this turns out to be unnecessary in our construction because we see that $\delta(t)$ will only have positive components.

Now define the following useful sets:

$$\text{PP}(t) := \{i \geq k(t) : D_i(p)(\mathbf{a}) = D_{u_i}(p)(\mathbf{a}) \neq 0\}.$$

Note that $\text{PP}(t)$ is a finite set. We partition $\text{PP}(t)$ into the sets $\text{PP}_1(t)$, $\text{PP}_2(t)$, ..., $\text{PP}_n(t)$, where for $\alpha = 1, \dots, n$,

$$\text{PP}_\alpha(t) := \{i \in \text{PP}(t) : \alpha \text{ is the smallest index s.t. } W_\alpha(u_i) > W_\alpha(u_k)\}.$$

To see that this is indeed a partion of $\text{PP}(t)$, recall that for all $i > k = k(t)$, $u_k <_A u_i$, and by the properties of the matrix W , this means there is some $\alpha = 1, \dots, n$ such that $W_\alpha(u_k) < W_\alpha(u_i)$ and $W_\beta(u_k) = W_\beta(u_i)$ for all $1 \leq \beta < \alpha$.

The goal (9) is now further reduced to showing this for each $\alpha = 1, \dots, n$:

$$\frac{1}{n} u_k(\delta(t)) c_k \cdot |D_k(p)(\mathbf{a})| > \sum_{j \in \text{PP}_\alpha(t)} u_j(\delta(t)) c_j \cdot |D_j(p)(\mathbf{a})|. \quad (10)$$

Now we show how to choose $\delta(t) = (\delta_1(t), \dots, \delta_n(t))$. To do this, we introduce an auxiliary sequence

$$\varepsilon(t) = (\varepsilon_1(t), \dots, \varepsilon_n(t)).$$

Then $\delta(t)$ is defined through

$$\delta_i(t) := \prod_{j=1}^n \varepsilon_j(t)^{w_{ji}}, \quad (11)$$

where w_{ji} are the entries of the matrix W above. If u is given by (6) then

$$\begin{aligned} u(\delta(t)) &= \prod_{i=1}^n \left(\prod_{j=1}^n \varepsilon_j(t)^{w_{ji}} \right)^{e_i} \\ &= \prod_{j=1}^n \prod_{i=1}^n \varepsilon_j(t)^{w_{ji} e_i} \\ &= \prod_{j=1}^n \varepsilon_j(t)^{W_j(u)}, \end{aligned}$$

where $W_j(u)$ is given by (1). Choose $K(t)$ so that $K(t)$ is monotonically increasing in t and

$$K(t) > \max \left\{ \left| \frac{c_j \cdot D_j(p)(\mathbf{a})}{c_k \cdot D_k(p)(\mathbf{a})} \right| : j \in \text{PP}(t) \right\}.$$

Now to show goal (10) it suffices to show (for $\alpha = 1, \dots, n$)

$$\frac{1}{n} > K(t) \sum_{j \in \text{PP}_\alpha(t)} \frac{u_j(\delta(t))}{u_k(\delta(t))} = K(t) \sum_{j \in \text{PP}_\alpha(t)} \prod_{\beta \geq \alpha} \varepsilon_\beta(t)^{W_\beta(u_j) - W_\beta(u_k)},$$

or,

$$\frac{1}{n} > K(t) \sum_{j \in \text{PP}_\alpha(t)} \varepsilon_\alpha(t)^{W_\alpha(u_j/u_k)} \prod_{\beta > \alpha} \varepsilon_\beta(t)^{W_\beta(u_j/u_k)},$$

since $W_\beta(u_j) - W_\beta(u_k) = W_\beta(u_j/u_k)$.

Next choose $L_*(t) > 0$ such that $L_*(t)$ is monotonically decreasing in t and

$$L_*(t) < \min\{W_\alpha(u_j) - W_\alpha(u_k) : j \in \text{PP}(t), W_\alpha(u_j) > W_\alpha(u_k), \alpha = 1, \dots, n\}.$$

Again, choose $L^*(t)$ such that $L^*(t)$ is monotonically increasing in t and

$$L^*(t) > \max\{W_\alpha(u_i) : i \in \text{PP}(t) \quad \text{and} \quad \alpha = 1, \dots, n\}.$$

Since we shall choose $\varepsilon_\alpha(t) < 1$ for all t and α , goal (10) further simplifies to

$$\frac{1}{n} > |\text{PP}(t)| K(t) \varepsilon_\alpha(t)^{L_*(t)} \prod_{\beta > \alpha} \varepsilon_\beta(t)^{-L^*(t)}. \quad (12)$$

To satisfy (12), we define $\varepsilon(t)$ using the equation

$$\frac{1}{2n} = |\text{PP}(t)| \cdot K(t) \varepsilon_\alpha(t)^{L_*(t)} \left(\prod_{\beta > \alpha} \varepsilon_\beta(t) \right)^{-L^*(t)}.$$

In other words,

$$\varepsilon_\alpha(t) = \left(\frac{1}{2n |\text{PP}(t)| \cdot K(t)} \right)^{1/L_*(t)} \left(\prod_{\beta > \alpha} \varepsilon_\beta(t) \right)^{L^*(t)/L_*(t)}.$$

This defines $\varepsilon_\alpha(t)$ in terms of those $\varepsilon_\beta(t)$ with larger subscripts ($\beta > \alpha$). In particular, $\varepsilon_n(t)$ is also defined. It is not hard to see that $1 > \varepsilon_\alpha(t) > 0$. Finally, $\varepsilon_\alpha(t)$ is monotonically decreasing in t (since $1/L_*(t)$, $L^*(t)$, $|\text{PP}(t)|$, and $K(t)$ are increasing in t).

We now prove that $\delta(t)$ defined via (11) has the desired properties. We have constructed $\delta(t)$ so as to make $\sigma(p_i) = \text{sign}(p_i(\mathbf{a} + \delta(t))) \neq 0$ (cf. (2) and (10)). It only remains to show that δ is an infinitesimal perturbation:

(i) For all $i = 1, \dots, n$, we want $\delta_i(t) \rightarrow 0_n$ as $t \rightarrow \infty$. Furthermore, we want $1 > \delta_i(t) > 0$ and $\delta_i(t)$ to be monotonically decreasing in t . These properties follow from our choice of $\delta_i(t)$ in terms of $\varepsilon_j(t)$.

(ii) We must show that for all $t \geq 1$ and for all $i \geq t$, $\text{sign}(p_i(\mathbf{a} + \delta(t))) = \text{sign}(p_i(\mathbf{a} + \delta(i)))$. This follows from two facts: (i) goal (10) still holds when we replace $\delta(t)$ by $\delta(i)$. (To see this: (10) follows from (12), but in (12) we may replace $\varepsilon_\beta(t)$ by $\varepsilon_\beta(i)$ for all β .) (ii) $\text{sign}(D_k(p_i)(\mathbf{a}) c_k u_k(\delta(t))) = \text{sign}(D_k(p_i)(\mathbf{a}) c_k u_k(\delta(i)))$, since c_k and $u_k(\delta(i))$ are positive.

This completes the proof of the consistency theorem.

Q.E.D.

4. APPLICATION

In what sense does the consistency theorem give a “geometrical justification” of our method? First we need to understand the nature of the *output* of a geometrical

algorithm. (We have already assumed its input is essentially numerical, i.e., a sequence of numbers.) We now assume that a geometrical algorithm outputs a combinatorial structure (abstract objects with relations, namely a hypergraph H) together with a labeling function λ on the hyperedges of H . More precisely, λ assigns a geometric object $\lambda(e)$ to each hyperedge e of H . These geometric objects (for instance, points or lines) are functions of the input parameters. What we want to clarify is the concept of a "perturbation" of (H, λ) .

For example, consider the input parameters $\mathbf{x} = (x_1, \dots, x_{2n})$ representing n points in the plane, with output combinatorial structure a graph G with labeling functions p and L (respectively) for the vertices and edges of G . G will represent a Voronoi diagram of the input points, and the label of a vertex v (resp. edge e) of G is a point p_v (resp. line L_e). These labels p_v , L_e are parameterized by the input parameters, $p_v = p_v(\mathbf{x})$ and $L_e = L_e(\mathbf{x})$. Suppose we perturb the input to $\mathbf{x} + \mathbf{d}$, and the resulting output is G^* with vertex and edge labels of the form $p_v^*(\mathbf{x} + \mathbf{d})$ and $L_e^*(\mathbf{x} + \mathbf{d})$. If we let the perturbation $\mathbf{d}$ approach, but never actually reach zero, then the graph G^* may stabilize (remain constant). The value of p^* and L^* may either converge or become undefined. In any case, provided G^* stabilizes, we call (G^*, p^*, L^*) a *perturbed output* (or, a *perturbed version of* (G, p, L)). We thus allow the possibility that p^* , L^* remain undefined on some parts of the structure G^* .

This definition can be generalized to other problems if desired. Note that G^* is a combinatorial structure that the algorithm will produce for some actual perturbation of the input. However, p^* and L^* are only the limits of respective sequences of actual labels; there may be no actual input that produces such labels.

The next figure shows (a) the Voronoi diagram G of a set of four co-circular points and (b) a perturbed version of G . In the perturbed version, we have two Voronoi vertices whose labels are identical (they are the same point). The label of the edge connecting them will be undefined. Clearly, there are basically two perturbed versions for the original Voronoi diagram. (Figs. 4.1 and 4.2).

THEOREM 4 (Perturbation). *Let A be any algorithm that satisfies our input/output assumptions, works correctly for non-degenerate inputs, and halts for all inputs. Then A , with our evaluation scheme, on degenerate input parameters will generate a perturbed output.*

Proof. Let $\mathbf{a}$ be the input parameters and let $(\delta(1), \delta(2), \dots)$ be the infinitesimal perturbation at the input $\mathbf{a}$ corresponding to our evaluation scheme. Since A always halt, it evaluates only a finite number of polynomials. Then for i sufficiently large, the perturbation $\delta(i)$ will make all these polynomials evaluate to non-zero, with exactly the same sign as that given by our evaluation scheme. The algorithm, being correct for non-degenerate inputs, compute an appropriate output (i.e., combinatorial structure plus labels) that depends only on the sequence of signs of the branch steps. It is not hard to see that this output is actually a perturbed output (it is the stabilization of a sequence of perturbed inputs). Q.E.D.

If an application is unhappy with perturbed outputs, we can often convert them

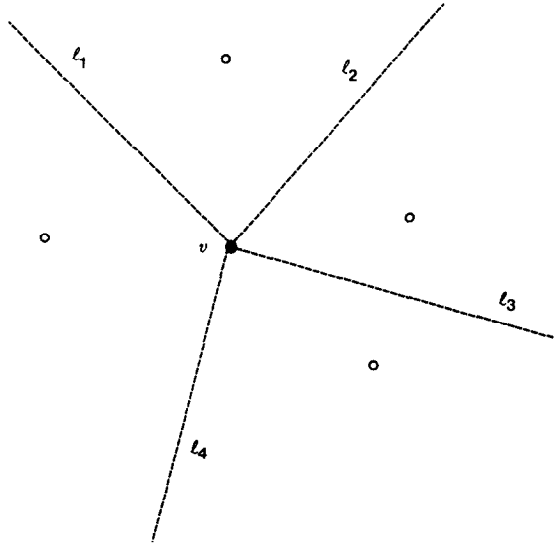


FIG. 4.1. Diagram of four co-circular points.

into the exact outputs. In the example of the Voronoi diagram, for instance, a single pass over the perturbed output suffices to detect those vertices with the same label. Such vertices can be identified and edges incident on them can be modified in the appropriate way.

In [13] we called the notion of degeneracy treated here *induced degeneracy* since

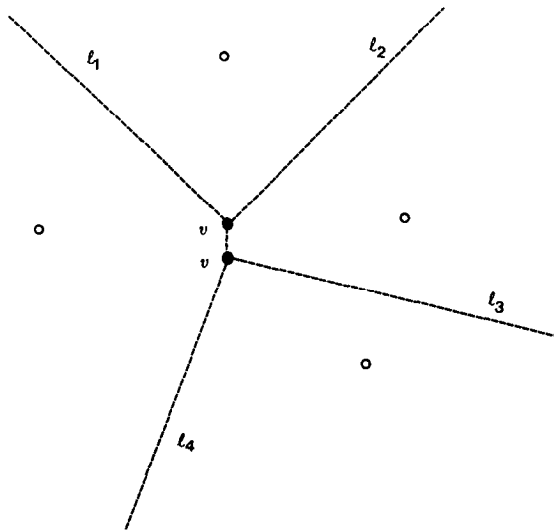


FIG. 4.2. Perturbed version.

it is induced by the algorithm in question. There is the corresponding notion of *inherent degeneracy* that is independent of the algorithm. One could define an input to be “inherently non-degenerate” if its output has no other perturbed version (except, of course, for the actual output).

5. RATIONAL FUNCTIONS

This section extends our method to the evaluation of rational functions. These might arise, for instance, in the solution of a system of linear equations, with the solutions expressed as a ratio of determinants. Another example is the linear programming problem.

Consider the evaluation of the rational function p/q at any $\mathbf{a} \in \mathcal{R}^n$. To design our blackbox, it suffices to give a sign only in case $q(\mathbf{a}) \neq 0$. The sequence $S(p/q)$ of partial derivatives of p/q can be constructed in the usual fashion, according to some fixed admissible ordering. The sign is then the sign of the first non-zero value obtained by evaluating the polynomials in $S(p/q)$ in order. If the sign of the first non-zero is positive, we write this as

$$p/q >_{\mathbf{a}} 0.$$

Let $\mathbf{R} = \mathcal{Q}[x_1, \dots, x_n]$ and $\mathbf{F} = \mathcal{Q}(x_1, \dots, x_n)$. For any $f \in \mathbf{F}$, we say f is *regular* at $\mathbf{a}$ if $f(\mathbf{a})$ is defined, i.e., if $f = p/q$, where $p, q \in \mathbf{R}$ then $q(\mathbf{a}) \neq 0$. It is not hard to see that if f is regular at $\mathbf{a}$ then so is every derivative of f . This is implied by the following lemma, which gives us much more information:

LEMMA 5. *Let $p, q \in \mathbf{R}$ and $u \in \text{PP}(x_1, \dots, x_n)$. Then $D_u(p/q)$ has the form*

$$\sum_{v|u} (D_v(p)/q) \cdot P_v$$

where each P_v is a multivariate polynomial with integer coefficients over the variables

$$\{D_w(q)/q : w|u\}.$$

Moreover, $P_u = 1$. Note that the notation $v|u$ means that v divides u , where $u, v \in \text{PP}$.

Proof. For any variable x , we have

$$D_x(p/q) = \frac{D_x(p)}{q} - \frac{p}{q} \cdot \frac{D_x(q)}{q}$$

which has the desired form with $P_1 = -D_x(q)/q$ and $P_x = 1$. Inductively, if $D_u(p/q)$ has the above form, then

$$\begin{aligned}
 D_{ux}(p/q) &= \sum_{v|u} D_x \left(\frac{D_v(p)}{q} \cdot P_v \right) \\
 &= \sum_{v|u} \left[D_x \left(\frac{D_v(p)}{q} \right) \cdot P_v + \frac{D_v(p)}{q} \cdot D_x(P_v) \right] \\
 &= \sum_{v|u} \left[\left(\frac{D_{xv}(p)}{q} - \frac{D_v(p)}{q} \cdot \frac{D_x(q)}{q} \right) \cdot P_v + \frac{D_v(p)}{q} \cdot D_x(P_v) \right] \\
 &= \sum_{v|u} \left[\frac{D_v(p)}{q} \cdot \left(-\frac{D_x(q)}{q} \cdot P_v + D_x(P_v) \right) + \frac{D_{xv}(p)}{q} \cdot P_v \right] \\
 &= \sum_{w|ux} \frac{D_w(p)}{q} \cdot P'_w,
 \end{aligned}$$

for suitable P'_w , after collecting terms with the same factor $D_w(p)/q$. It is seen that P'_w has the correct form and P'_{ux} is in fact equal to $P_u = 1$. Q.E.D.

LEMMA 6. *Let p/q be regular at $\mathbf{a}$. Then $p/q >_{\mathbf{a}} 0$ if and only if either $[p >_{\mathbf{a}} 0$ and $q(\mathbf{a}) > 0]$ or $[p <_{\mathbf{a}} 0$ and $q(\mathbf{a}) < 0]$.*

Proof. Using the previous lemma, we see that $p/q >_{\mathbf{a}} 0$ if and only if there is a power product u such that $D_u(p)/q(\mathbf{a}) > 0$ and $D_v(p)/q(\mathbf{a}) = 0$ for all v that properly divides u . The desired result follows at once. Q.E.D.

THEOREM 7. *The set $\mathbf{F}_{\mathbf{a}} \subseteq \mathbf{F}$ of regular functions at $\mathbf{a}$ forms an ordered ring with respect to the ordering $>_{\mathbf{a}}$. Moreover, the ordering $>_{\mathbf{a}}$ on $\mathbf{F}_{\mathbf{a}}$ extends the original ordering on $\mathbf{R} \subseteq \mathbf{F}_{\mathbf{a}}$ induced by evaluation at $\mathbf{a}$.*

Proof. Let $\sigma: \mathbf{F}_{\mathbf{a}} \rightarrow \{-1, 0, +1\}$ be the function defined by $\sigma(f) = 0$ if and only if $f = 0$ and for $f \neq 0$, $\sigma(f) = +1$ if and only if $f >_{\mathbf{a}} 0$. Using the characterization of the ordering $>_{\mathbf{a}}$ given in the previous lemma, we can check that σ is a total sign function, similar to the proof in [13] in the case of $\mathbf{R}$. Q.E.D.

6. FINAL REMARKS

Let us briefly remark on the relation of our scheme to other published methods for removing degeneracies.

(i) It is not hard to see that our method subsumes and simplifies Edelsbrunner's simulation of simplicity.

(ii) One may verify that the scheme proposed in [2] to break ties in computing distances between pairs of points is also a special case of ours.

(iii) Our consistency proof requires that the polynomial ring $\mathbf{R}$ be denumerable. Is this restriction essential?

(iv) Using the extension of our method to rational functions, we also have a family of symmetry breaking rules for the simplex algorithms.

However, we have not been able establish any simple connection between our method and the known rules for symmetry breaking rules for simplex algorithms. Moreover, our rule is hardly practical in this setting, and nowhere matches the simplicity of a rule due to Bland (see [10]). On the other hand, we observe that no consistency proof has ever been given for Bland's rule (the usual justification simply shows that the method does not cycle—it is not obvious that there is an actual perturbation of the input that gives rise to the same sequence of decisions).

In some applications, the user would like some control over the perturbation. Mehlhorn² describes a problem that required controlled perturbation. The problem is to test if two polytopes intersect and we want to regard a vertex touching the relative interior of a face of the other polytope to be degenerate (there are other types of degeneracies such as vertex touching an edge; perhaps we need not worry whether such intersections are reported). Under this condition, the perturbation must move the vertices of the polytopes in an outward direction in order not to miss any intersection. See [13] for another instance. It would be useful to devise perturbation schemes to satisfy such restriction. Our method here does not lend itself easily to meaningful control.

Finally, we show that our scheme is not complete with respect to infinitesimal perturbations. Mike Paterson originally constructed a counterexample over three variables x, y, z which we describe here. For $t = 1, 2, \dots$, let $\delta(t) = (\delta_1(t), \delta_2(t), \delta_3(t))$, where

$$\delta_i = e^{-e^{et} + e^{-it}}$$

for $i = 1, 2, 3$. One checks that this is a perturbation at the origin $(x, y, z) = (0, 0, 0)$, and the ordering $<'$ it induces on polynomials satisfies

$$x >' y, \quad x >' z, \quad x <' y + z.$$

But in any admissible ordering, the first two inequality implies $x >' y + z$.

To construct a somewhat simpler example for two variables x, y , we exploit the following property: in any total ordering $<'$ on polynomials induced by admissible orderings, it is clear that if $x <' y$ then for any positive coefficient c , $cx <' y$. We construct an ordering induced by infinitesimal perturbations where this fails. Let $\delta(t) = (\delta_x(t), \delta_y(t))$ be given by

$$\delta_x(t) = e^{-t}, \quad \delta_y(t) = (1 + t^{-1}) e^{-t}.$$

We check that this is an infinitesimal perturbation at $(0, 0)$: for any polynomial $p(x, y)$, the sign of p induced by $\delta(t)$ depends only on its lowest degree monomials.

² Private communication.

So let the lowest degree monomials in $p(x, y)$ be

$$\sum_{i=1}^k c_i x^{d-d_i} y^{d_i},$$

where $c_1, \dots, c_k$ are non-zero constants and $0 \leq d_1 < d_2 < \dots < d_k \leq d$. Evaluating at $\delta(t)$ gives

$$x^d \left(\sum_{i=1}^k c_i (1+t^{-1})^{d_i} \right) = x^d \sum_{j=0}^d t^{-j} \left(\sum_{i=1}^k c_i \binom{d_i}{j} \right).$$

The sign of $p(x, y)$ is determined by the sign of the first non-zero coefficient of t^{-j} for $j=0, \dots, d$. We claim that the first k coefficient cannot all be zero. Otherwise, this is equivalent to

$$\begin{pmatrix} 1 & 1 & \dots & 1 \\ d_1 & d_2 & \dots & d_k \\ d_1^2 & d_2^2 & \dots & d_k^2 \\ \dots & & & \dots \\ d_1^{k-1} & d_2^{k-1} & \dots & d_k^{k-1} \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_k \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}.$$

Since Vandermonde's matrix is non-singular, this implies that all the c_i 's are zero, contradiction. This shows that we have a perturbation at $(0, 0)$. In particular, note that if $<'$ is the ordering on polynomials induced by this perturbation then $x <' y$, but for any constant $c > 1$, we have $cx >' y$. This concludes our example.

This example opens up other questions. In particular, it seems possible to extend the class of orderings induced by admissible orderings so as to get a completeness theorem. Another important direction is to treat algebraic functions such as radicals. This would bring shortest path problems within the scope of symbolic perturbation methods.

ACKNOWLEDGMENTS

I am grateful for the hospitality of Professor Mehlhorn in Saarbruecken, where this work was carried out. This work has benefitted from our discussions with Mike Paterson.

REFERENCES

1. B. BUCHRERGER, Gröbner Bases: An algorithmic method in polynomial ideal theory, in "Recent Trends in Multidimensional Systems Theory" (N. K. Bose, Ed.), Reidel, Dordrecht, 1985.
2. C. MONMA, M. PATERSON, S. SURI, AND F. F. YAO, Computing Euclidean maximum spanning trees, in "4th ACM Symp. on Comput. Geom., 1988," pp. 241-251.
3. A. CHARNES, Optimality and degeneracy in linear programming, *Econometrica* **20**, No. 2 (1952), 160-170.

4. V. CHVÁTAL, "Linear Programming," Freeman, San Francisco, 1983.
5. T. DUBÉ, B. MISHRA, AND C. K. YAP, "Admissible Orderings and Bounds for Gröbner Bases Normal Form Algorithm," Report 88, NYU-Courant Robotics Lab, 1986.
6. H. EDELSBRUNNER, Edge-skeletons in arrangements with applications, *Algorithmica* 1 (1986), 93–110.
7. H. EDELSBRUNNER AND E. P. MÜCKE, Simulation of simplicity: A technique to cope with degenerate cases in geometric algorithms, in "4th ACM Symp. on Computational Geometry, 1988," pp. 118–133.
8. H. EDELSBRUNNER AND R. WAUPOTITSCH, Computing a ham-sandwich cut in two dimensions, *J. Symbolic Comput.* 2 (1986), 171–178.
9. B. MISHRA AND C. K. YAP, "Notes on Gröbner Bases," Report 87, NYU-Courant Robotics Lab, 1986; *J. Inform. Sci.*, in press.
10. C. PAPADIMITRIOU AND K. STEIGLITZ, "Combinatorial Optimization," Prentice-Hall, Englewood Cliffs, NJ, 1982.
11. L. ROBBIANO, On the theory of graded structures, *J. Symbolic Comput.* 2 (1986), 139–170.
12. L. ROBBIANO, Term orderings on the polynomial ring, in "EUROCAL '85, Lecture Notes in Computer Science, No. 20, 1985," pp. 513–517.
13. C. K. YAP, Symbolic treatment of geometric degeneracies, in "Lecture Notes in Control and Information Sciences, Vol. 113," pp. 348–358, Springer-Verlag, New York/Berlin, 1988; *J. Symbolic Comput.*, in press.